SPIDER-MAN 2D — TECNOLOGÍAS, MECÁNICAS Y SISTEMA DE SPRITES

Documento técnico: qué tecnologías se usaron (incluyendo Python y Phaser 3) y cómo funciona cada mecánica del juego

1. RESUMEN GENERAL DE TECNOLOGÍAS UTILIZADAS

Tecnología / Herramienta	Dónde se usa	Para qué función principal se utilizó
Python 3 (Pillow + NumPy + SciPy)	`tools/slice_sprites.py` `tools/build_hero_assets.py` `tools/make_art.py`	Procesamiento de arte y sprites: recorta automáticamente las hojas de sprites (*sprite sheets*), borra el fondo a transparente, centra a Spider-Man en celdas de `64×72 px` y dibuja las texturas pixel-art de edificios y fondos.
HTML5 + CSS3	`frontend/index.html` `frontend/style.css`	Estructura la ventana del juego (`#game`) y maqueta las pantallas superpuestas (modales de inicio y derrota/victoria), la guía de controles, el formulario y la tabla con estética de cómic.
JavaScript (ES6+)	`frontend/src/*.js`	Lenguaje principal de toda la lógica del juego: cálculos matemáticos, física vectorial del péndulo, máquinas de estados de personajes, temporizadores, teclado/mouse y peticiones `fetch()`.
Phaser 3 (v3.80.1) (Motor 2D WebGL/Canvas)	Todo el `frontend/`	Provee el motor gráfico (WebGL), el motor de física (**Arcade Physics**), el sistema de cámaras (**Camera2D**), las animaciones interpoladas (**Tweens**) y el administrador de texturas y sprites.
PHP 8 + PDO	`backend/api/puntajes/` `backend/config/`	Construye la API REST que recibe los puntajes en JSON, valida el tope máximo de 40.000 puntos y ejecuta consultas preparadas seguras contra la base de datos.
SQL (MariaDB / MySQL) + XAMPP	`database/schema.sql`	Almacena de forma persistente los puntajes en la tabla `puntajes` y resuelve el ranking ordenando con `ORDER BY puntos DESC, tiempo_segundos ASC`.

2. CÓMO FUNCIONAN LOS SPRITES (DE PYTHON AL JUEGO EN VIVO)

¿Qué es un Sprite y cómo logra moverse? Un sprite es una imagen 2D con fondo transparente (`.png`) que muestra una pose de un personaje. Para que parezca que camina, salta o golpea, el juego funciona igual que un **dibujo animado o folioscopio (flipbook)**: muestra varias fotos seguidas muy rápido (**8 fotos por segundo**).

A.1. Etapa Previa: Recorte y Preparación de Sprites con Python

Python 3 + Pillow (PIL) + NumPy + SciPy

Originalmente, los dibujos de los personajes venían juntos en grandes láminas llamadas **hojas de sprites (sprite sheets)** con un color de fondo liso. Para procesarlos de forma exacta se usaron 3 scripts en **Python**:

- **1. Recorte automático y fondo transparente (`tools/slice_sprites.py`):**
  - Abre la hoja de sprites con `PIL.Image` y la transforma en una matriz numérica `RGBA` con `NumPy`.
  - Lee el píxel de la esquina (`a[2,2,:3]`) para detectar el color de fondo y convierte todos los píxeles de ese color en **transparentes** (`(0, 0, 0, 0)`).
  - Aplica dilatación morfológica y etiquetado de objetos con `scipy.ndimage` (`binary_dilation` + `label` + `find_objects`) para detectar automáticamente cada figura, recortarla sin perder partes sueltas y guardarla como un archivo PNG individual. Así se obtuvieron y clasificaron las **440 imágenes PNG** de los 10 enemigos y 9 jefes dentro de `frontend/assets/sprites/`.
- **2. Alineación de Spider-Man en celdas uniformes (`tools/build_hero_assets.py`):**
  - Toma cada cuadro recortado de Spider-Man, calcula su centro horizontal con `np.nonzero(alpha > 0)` y lo pega dentro de un lienzo uniforme de `64×72 píxeles`.
  - En las poses de suelo (`hero_idle`, `hero_run0..5`, `hero_punch0..2`) apoya los pies exactamente en el borde inferior (`CH - im.height`); en las poses aéreas (`hero_jump`, `hero_swing`, `hero_climb`) lo centra a altura `AIR_Y = 48` para que el personaje nunca "salte" ni tiemble al cambiar de imagen.
- **3. Generación procedural de texturas pixel-art (`tools/make_art.py`):**
  - Usa `NumPy` y `Pillow` para dibujar por código las texturas repetibles sin costuras de las fachadas de edificios (`win_tower.png`, `win_brick.png`, `win_ware.png` de `256×256 px`), cielo, luna, capas lejanas y los sprites de respaldo (usados para *Lizard*).

A.2. Etapa en Vivo: Cómo Funcionan los Sprites Dentro de la Página

Phaser Loader + Texture Manager + Arcade Body (assets_data.js / enemies.js)

- **1. El Mapa / Catálogo de Sprites (`SPRITE_MANIFEST` en `assets_data.js`):** Como sería muy difícil escribir el nombre de 440 archivos a mano en la lógica de combate, `assets_data.js` le asigna un "apodo" corto a cada imagen PNG (ej. `spr_boss_kingpin_walk_0`) y agrupa los cuadros según la acción: quieto (`idleKeys`), caminar (`walkKeys`), atacar (`atkKeys`), embestir (`chargeKeys`) o daño (`hitKeys`).
- **2. Precarga en Memoria y Filtro Pixel-Art (`main.js`):** Durante `preload()`, Phaser carga todas las imágenes en memoria con `this.load.image(clave, ruta)` y aplica el filtro `Phaser.Textures.FilterMode.NEAREST` para que al escalar los sprites conserven sus píxeles nítidos sin verse borrosos.
- **3. Cambio Automático de Foto según la Acción (`player.js` y `enemies.js`):** Un contador de tiempo (`d.animT += dt`) calcula qué cuadro mostrar a 8 FPS (`Math.floor(d.animT * 8) % frames.length`), cambiando la textura con `setTexture(key)` según lo que esté haciendo el personaje. Si mira hacia la izquierda, espeja la imagen con `setFlipX(true)`.
- **4. Punto de Apoyo en los Pies (Corrección de Piso):** Todos los personajes tienen su anclaje en la planta de los pies con `setOrigin(0.5, 1.0)` (centro horizontal, borde inferior en los pies) y al cambiar de foto recalibran su caja física (`body.setSize` y `body.setOffset`). Así, aunque cambie la altura del dibujo en cada animación, **los pies siempre quedan apoyados sobre la calle (`Y = 1400`)**.

3. TECNOLOGÍAS APLICADAS A CADA MECÁNICA ESPECÍFICA

B. Movimiento del Personaje y Balanceo en Telaraña

Phaser Arcade Physics + Trigonometría JS (player.js)

- **Carrera, Salto y Gravedad:** Se usa el motor **Arcade Physics** de Phaser (`this.sp.body`). La gravedad global (`GRAV: 1500`) empuja a Spider-Man hacia abajo, mientras que al apretar `A/D` o `Espacio` se modifican las velocidades horizontales (`setVelocityX`) y verticales (`setVelocityY(-640)`).
- **Trepar y Saltar por Paredes (Parkour):** El motor físico detecta colisiones laterales mediante las banderas booleanas `body.blocked.left` y `body.blocked.right`. Si toca una pared en el aire y se presiona `W`, sube a velocidad constante (`-240 px/s`).
- **Balanceo con Telaraña (Clic Derecho):** Combina el puntero de Phaser (`activePointer.rightButtonDown()`) con **física vectorial de péndulo en JavaScript**: calcula la distancia entre Spider-Man y el punto del edificio con `Math.hypot(dx, dy)`. Cuando el héroe supera el largo de la cuerda (`w.len`), corrige su posición al radio del círculo y anula la velocidad radial usando producto escalar, transformando la caída en impulso de balanceo. Además, rota el sprite con `Math.atan2()`.
- **Esquiva / Dash (Tecla Shift):** Desactiva temporalmente la gravedad del cuerpo (`body.allowGravity = false` durante 0.18s), aplica un impulso horizontal de `720 px/s` y activa una variable de invulnerabilidad (`this.inv = 0.35`).

C. Creación del Mapa y Generación del Mapa Infinito

JavaScript Map + RandomDataGenerator + Graphics (city.js)

- **División en Bloques (Chunks de 1.000 px):** El mapa infinito no existe todo al mismo tiempo (porque llenaría la memoria RAM). Se divide en tramos de `1000 px` de ancho llamados *chunks*, administrados con una estructura `new Map()` de JavaScript.
- **Carga y Liberación Dinámica:** En cada fotograma se calcula en qué bloque está Spider-Man con `ci = Math.floor(px / 1000)`. El código crea automáticamente 3 bloques hacia adelante (`ci + 3`) y destruye de la memoria (`destroy()`) los bloques que quedaron más de 3 tramos atrás (`i < ci - 3`).
- **Generación Procedural con Semilla:** Usa `Phaser.Math.RandomDataGenerator` con una semilla por bloque para decidir al azar la altura, ancho y estilo de cada edificio, las farolas y los autos estacionados.
- **Superficies de Colisión y Enganche:** El suelo de la calle se agrega a un grupo físico estático (`physics.add.staticGroup()`) en `Y = 1400`, y las coordenadas de cada edificio se guardan en un arreglo de rectángulos (`c.rects`) para que la mira sepa dónde puede engancharse la telaraña.

D. Desplazamiento del Mapa y Efecto de Profundidad

Phaser Camera2D + TileSprites Parallax (main.js / city.js)

- **Seguimiento Suave de Cámara:** Se utiliza `cameras.main.startFollow(this.player.sp, true, 0.09, 0.09)`. El valor `0.09` es una interpolación lineal (*lerp*) que hace que la cámara siga a Spider-Man con suavidad cinematográfica en lugar de moverse rígidamente.
- **Adelanto de Cámara según Velocidad:** Con `setFollowOffset()`, cuando Spider-Man se balancea rápido hacia la derecha, la cámara se adelanta levemente para mostrar más calle hacia adelante.
- **Efecto Parallax Multicapa:** Los edificios lejanos usan `Phaser.GameObjects.TileSprite` fijados a la pantalla (`setScrollFactor(0)`) cuya textura interna se desplaza a menor velocidad que la cámara (`scrollX * 0.08` para el fondo lejano y `scrollX * 0.2` para el fondo medio), creando sensación de profundidad 3D en un juego 2D.

E. Sistema de Ataques (Golpes y Telaraña a Enemigos)

Geometría de Cajas Físicas + Tweens (player.js / main.js)

- **Golpe Cuerpo a Cuerpo (Clic Izquierdo):** Utiliza un sistema de combo de 3 golpes (`this.combo = this.combo % 3 + 1`). En lugar de crear un objeto visible gris, calcula matemáticamente qué enemigos están delante de Spider-Man a menos de `85 px` en X y a menos de `75 px` de distancia vertical entre sus centros físicos (`body.center.y`). El 3º golpe del combo aplica más daño y mayor empuje (*knockback*).
- **Ataque con Telaraña a Distancia (Tecla F):** Convierte la posición del mouse en coordenadas del mundo (`cameras.main.getWorldPoint`) y busca con `Math.hypot()` al enemigo más cercano al cursor dentro de un alcance de `550 px`. Le aplica un estado de parálisis (`e.d.stun = 2` segundos), le superpone los sprites de red (`fx_web0`) y le aplica velocidad hacia Spider-Man para atraerlo.

F. Comportamiento de los Enemigos y Jefes (IA)

Máquinas de Estados en JS + Trigonometría (enemies.js)

- **Enemigos Cuerpo a Cuerpo (Duke, Slim, Spike, etc.):** Calculan en cada frame la dirección hacia Spider-Man (`Math.sign(p.x - e.x)`), caminan hasta ponerse a rango de golpe (`~42 px`) y atacan respetando su tiempo de recarga (`cd`). Si encuentran un obstáculo, saltan automáticamente.
- **Enemigos a Distancia (Blade y Moose):** Mantienen una distancia táctica: si Spider-Man se acerca a menos de `200 px`, retroceden; si está a rango medio, apuntan con `Math.atan2(dy, dx)` y disparan proyectiles.
- **Jefes Terrestres (Kingpin, Scorpion, Lizard, Electro, Sandman, Venom):** Alternan por tiempo entre 3 estados: `move` (perseguir y golpear), `wind` (preparación donde se ponen rojos por 0.6s avisando el ataque) y `charge` (embestida a 2.4x de velocidad; Electro además dispara un abanico de 5 rayos).
- **Jefes Voladores (Hobgoblin, Duende Verde, Doctor Doom):** Desactivan su gravedad (`allowGravity = false`), flotan en el aire siguiendo una onda senoidal (`Math.sin(d.flyT) * 45`), disparan calabazas o ráfagas de energía triples y realizan vuelos en picada (`charge`).

G. Cura del Botiquín y Reaparición cada 45 Segundos

Phaser Graphics + Sine Tweens + Delta Time (textures.js / city.js)

- **Creación y Posición Fija:** La textura del botiquín se dibuja por código con `Phaser.GameObjects.Graphics` en `textures.js`. Al crearse cada bloque de ciudad (*chunk*), se coloca exactamente 1 botiquín en la mitad del bloque (`medX = x0 + 500`) con un efecto de levitación continua usando `s.tweens.add({ ease: 'Sine.easeInOut', yoyo: true, repeat: -1 })`.
- **Curación al 100% y Cooldown de 45s:** Cuando la distancia entre Spider-Man y el botiquín es menor a `45 px` (`Math.hypot < 45`), restaura toda la vida (`p.hp = CFG.PLAYER_HP`), oculta el botiquín (`setVisible(false)`) e inicia su cuenta regresiva individual (`m.cooldown = 45`). En cada frame se descuenta el tiempo transcurrido (`m.cooldown -= dt`) hasta llegar a 0 y volver a aparecer.

H. Limitación del Mapa a 2 Pantallas Durante el Jefe

World Bounds + Camera Bounds + Math.Clamp (main.js / player.js / enemies.js)

- **Cálculo de las 2 Pantallas (2.560 px):** Como cada pantalla mide `1280 px` de ancho (`CFG.W`), al alcanzar el puntaje de un jefe la función `startBossFight()` calcula una arena cerrada de exactamente el doble: `arenaW = 2560` píxeles alrededor de la posición actual del jugador (`arenaBounds = { minX, maxX }`).
- **Triple Bloqueo Físico y Visual:**
  1. **Cámara:** `cameras.main.setBounds(minX, -2500, 2560, ...)` impide que la vista salga de esas 2 pantallas.
  2. **Mundo Físico:** `physics.world.setBounds(minX, -3000, 2560, ...)` coloca paredes físicas invisibles en los bordes.
  3. **Restricción Matemática:** Tanto en `player.js` como en `enemies.js` se aplica `Phaser.Math.Clamp(x, minX, maxX)` para garantizar que ni Spider-Man ni el jefe puedan salirse de la arena ni siquiera durante un dash o embestida.
- **Barreras de Energía y Pausa de Enemigos Comunes:** Se dibujan dos columnas láser rojas parpadeantes en `minX` y `maxX` con `Phaser.Graphics`, se eliminan los enemigos comunes presentes y se bloquea `spawnEvent()` hasta que el jefe muere y `endBossFight()` vuelve a abrir el mapa infinito.